

Photo Booth Integrations Guide:

Webhooks, Triggers, API & Payments

How to connect a BoothLedger photo booth to the world around it: lights and relays that react to each session, signed webhooks to Zapier, Make or your own server, a local API for terminals and scripts, and pay to play with card, coin and QR payments. Every step uses the real menu names, with copy-paste examples.

Last updated 4 October 2026 · Integrations are part of the AI Photobooth plan and the free 30-day trial.

ON THIS PAGE

1. Overview

[Which one do I need?](#)

[Plans & internet](#)

2. Session moments

3. Triggers (lights & relays)

[Set up a trigger](#)

[How the booth calls](#)

[Worked examples](#)

4. Webhooks

[Add a webhook](#)

[Delivery & retries](#)

[What BoothLedger sends](#)

[Verify the signature](#)

[Zapier & Make recipes](#)

5. Control API

[Endpoints](#)

[Examples](#)

6. Pay to play

[Packages & bundles](#)

[Card reader & coins](#)

[QR payments \(Stripe\)](#)

[PIN protection](#)

7. Payments, fees & invoices

[Fees & VAT](#)

[Invoices](#)

8. Troubleshooting

1. Overview

BoothLedger integrations have four building blocks. They can be used on their own or together.

[Booth → devices at the venue](#)

Triggers

The booth calls a web address on the venue network at moments in a session: WLED lights, Shelly relays, smart plugs, Home Assistant. No internet needed.

[BoothLedger → internet services](#)

Webhooks

BoothLedger posts signed JSON to Zapier, Make, a CRM or your own server, including a share link for every uploaded photo.

[Your devices → the booth](#)

Control API

A local API on the booth PC: check status, start a session, lock and unlock, add paid credits. For terminals, scripts, Stream Decks and phone shortcuts.

[Guests → the booth](#)

Pay to play

Guests pay before each session by card reader, coins or a QR code on screen (Stripe). Sell single goes or packages like Photo + Boomerang.

Which one do I need?

I want to...	Use
Turn a ring light white on the countdown, fire a bubble machine, dim the room	Triggers
Log every photo to Google Sheets, ping Slack, update a CRM, email a link	Webhooks
Start a session from a phone or Stream Deck, pause the booth between bookings	Control API
Add credits from my own till, kiosk or payment bridge	Control API (/v1/credits/add)
Charge guests before each session	Pay to play
Call my own server on the venue network with signed JSON	Triggers (Signed message)
Call my own server on the internet	Webhooks

Plans & internet

Triggers, webhooks, the control API and pay to play are included in the **AI Photobooth plan** and the **free 30-day trial**. They are not part of the Basic plan. Before your first event with pay to play or QR payments, run a test payment at home or in the office so you know everything is set up right.

Feature	Needs internet?
Triggers	No. The booth calls devices on the venue network directly.
Control API	No. It runs on the booth PC.
Pay to play by card reader or coins	No.
Pay to play by QR code	Yes. The QR only shows while the booth PC is online.
Webhooks	BoothLedger sends them, so they reach your services even while the booth is offline. The booth keeps its session moments and forwards them once it is back online (see Delivery).

Some features in this guide (packages, ways to pay, QR payments and PIN protection) need the latest version of the BoothLedger app on the booth PC. If a setting described here is missing on your booth, update the app first.

2. Session moments

Triggers and webhooks fire on **moments** in a guest session. You pick the moments per trigger or per webhook. The names in the first column are what your device or server receives in the `type` field; the second column is the label you tick in BoothLedger.

Moment (type)	Label	When it fires	Key data fields
<code>session.start</code>	Session starts	A guest starts a session (after any design or background pick)	<code>mode</code> (<code>photo</code> , <code>boomerang</code> , <code>video</code> , <code>import</code>), <code>photo_count</code> ; with packages also <code>package</code> and <code>bundle_part</code> (1, 2...)
<code>countdown.started</code>	Countdown starts	Each countdown begins (once per photo)	<code>index</code> , <code>total</code> , <code>seconds</code>
<code>photo.captured</code>	Photo taken	Each photo is taken	<code>index</code> , <code>total</code>
<code>recording.started</code>	Recording starts	A boomerang or video starts recording	<code>mode</code>
<code>recording.stopped</code>	Recording stops	The recording ends	<code>mode</code>
<code>processing.done</code>	Photo ready (on the booth)	The result is saved on the booth	<code>mode</code>
<code>preview.shown</code>	Preview shown	The final preview screen opens	—
<code>print.submitted</code>	Print sent	A print is sent to the printer	<code>copies</code> , <code>auto</code> , <code>success</code>
<code>share.sent</code>	Shared (email, SMS or QR)	The guest shares by email, SMS/WhatsApp or opens the QR	<code>channel</code> (<code>email</code> , <code>sms</code> , <code>whatsapp</code> , <code>qr</code>), <code>status</code> (<code>sent</code> , <code>queued</code> , <code>failed</code> , <code>shown</code>)
<code>session.ended</code>	Session ends	The session is over	<code>reason</code> (<code>done</code> , <code>cancelled</code> , <code>error</code> , <code>abandoned</code> , <code>superseded</code>), <code>photos</code>

Moment (type)	Label	When it fires	Key data fields
credit.added	Credit added (pay to play)	A payment credit arrives at the booth	amount , source (e.g. key , stripe_qr , operator , or the source sent to the API), balance
photo.uploaded	Photo uploaded to BoothLedger (with share link)	Webhooks only. A photo or video reaches BoothLedger	photo_id , share_url , media (image or video)

Moments never contain guest contact details or file paths. Test sends use their own types: test from a booth trigger's **Send test**, and webhook.test from a webhook's **Send test**.

3. Triggers: lights, relays & local devices

A trigger is a web address, a method and the moments it fires on. The booth calls the address itself over the venue network, so triggers keep working with no internet. Each event can have up to **20 triggers**.

Set up a trigger on the booth

- 1 On the booth PC open **Settings > Integrations** and find **Triggers**. Switch **Triggers on**.
- 2 Tap **Add a trigger**. Under *What do you want to connect?* pick **WLED lights**, **Home Assistant**, **Shelly relay** or **Other web address**. Each preset fills in an example address and moments.
- 3 Give it a **Name** and type the device's **Address**. It must start with `http://` or `https://` and can't contain a username or password.
- 4 Under **How to call it** choose **Simple call (GET)** for lights and relays, or **Signed message (POST)** for Home Assistant and your own receivers.
- 5 Under **Call it when**, tick the moments, for example *Countdown starts* and *Session ends*.
- 6 For a signed message, create a **Signing secret** (*Create*, then *Show* and *Copy*) and paste it into your receiver.
- 7 Tap **Send test**. You'll see *Test sent — the device answered in ... ms* or the reason it failed. Then save. The trigger list shows *Working* or *Failing* for each trigger.

Or set it up on BoothLedger

Triggers belong to the event, so you can also set them up from any computer: open the event in BoothLedger, go to the **Integrations** tab and use the **Triggers** card.

- 1 Switch on **Triggers enabled**.
- 2 Click **+ WLED**, **+ Home Assistant**, **+ Shelly relay** or **+ Other web address**.
- 3 Fill in the name, choose **GET** or **POST (signed JSON)**, enter the address and tick the moments under **Call it when**.
- 4 Click **Save integrations**. The booth picks the change up within about 45 seconds.

Two things only happen on the booth: *Send test*, and signing secrets. Secrets are made on the booth, kept in Windows secure storage and never uploaded to BoothLedger. A secret is tied to the device (host and port) it was made for: if the address is later changed to a different device, for example from the web, the booth sends the calls **unsigned** until you create a new secret. The *Add the moment to the address* option is also set on the booth.

How the booth calls your device

Simple call (GET)

The booth opens the address once. It is **not retried**: switching a relay twice is worse than missing one cue. Turn on **Add the moment to the address** to append `event=<type>&session=<id>`, so one address can handle every moment:

```
http://192.168.1.80/booth?event=session.started&session=7f2c...
```

Signed message (POST)

The booth sends a JSON body with these headers: `Content-Type: application/json`, `User-Agent: BoothLedger-Booth`, `X-BoothLedger-Event` (the moment), `X-BoothLedger-Delivery` (an id that stays the same on retries) and, with a signing secret, `X-BoothLedger-Signature`.

Body of a booth trigger

```
{
  "v": 1,
  "id": "<delivery id>",
  "type": "session.started",
  "at": "2026-12-12T18:00:00.000Z",
  "event": { "id": "<booth event id>", "cloud_id": "<BoothLedger event id>" },
  "session": "<session id>",
  "test": false,
  "data": { "mode": "photo", "photo_count": 3 }
}
```

The signature works exactly like a webhook signature, so the same code verifies both: see [Verify the signature](#). Use the trigger's own secret from the booth.

Limits

- Every call times out after **3 seconds**. A 2xx answer counts as success.
- A POST is tried up to **3 times** (retries after 1 and 3 seconds) on a timeout, network error or 5xx. A 3xx or 4xx answer is final, and redirects are never followed.
- At most 4 calls run at once, and a cue that has waited more than 30 seconds is dropped, so lights never fire late.
- Send test bypasses the *Triggers on* switch, so you can check a device before you turn triggers on.

Tip: give each device a fixed address. Reserve its IP in the venue router (a "DHCP reservation") or set a static IP on the device, then use that IP in the trigger. Your devices and the booth PC must be on the same network.

Worked examples

A. WLED ring light: bright white on the countdown, back to a party preset afterwards

In WLED, first save the look you want between sessions as a preset (for example preset 2, a rainbow effect). Then add two triggers on the booth, both **WLED lights** with **Simple call (GET)**:

Name	Address	Call it when
Ring light white	<code>http://192.168.1.50/win&T=1&A=255&R=255&G=255&B=255&W=255&FX=0</code>	Countdown starts
Ring light party	<code>http://192.168.1.50/win&PL=2</code>	Session ends

In WLED's HTTP API, `T=1` turns the light on, `A` is brightness (0–255), `R G B W` are the colour channels (leave out `W` on an RGB-only strip), `FX=0` is the solid effect and `PL=2` loads preset 2.

B. Shelly relay: a 5-second pulse on every photo (bubble machine, confetti, spotlight)

Add a **Shelly relay** trigger with **Simple call (GET)** and tick *Photo taken*. For a Shelly Plus / Pro (Gen2 or newer) device:

```
http://192.168.1.60/rpc/Switch.Set?id=0&on=true&toggle_after=5
```

`toggle_after=5` makes the relay switch itself back off after 5 seconds, so it never stays on if the booth goes quiet. On an older Gen1 Shelly use `http://192.168.1.60/relay/0?turn=on&timer=5`. Change the number for a shorter or longer burst. Follow the equipment maker's instructions for wiring a relay to a bubble machine or confetti launcher, and make sure the relay is rated for the load.

C. Home Assistant: dim the room lights during a session

- 1 On the booth add a **Home Assistant** trigger. It uses **Signed message (POST)** and ticks *Session starts* and *Session ends*.
- 2 Set the address to your Home Assistant webhook, using the IP of your Home Assistant box and a webhook id that is hard to guess:
`http://192.168.1.20:8123/api/webhook/photobooth-k7q2x9`
- 3 In Home Assistant create this automation (Settings > Automations > Create > Edit in YAML). Change `light.function_room` to your own light.

Home Assistant automation (2024.10 or newer)

```
alias: Photo booth - dim the room during a session
mode: queued
triggers:
  - trigger: webhook
```

```

webhook_id: photobooth-k7q2x9
allowed_methods: [POST]
local_only: true
actions:
  - choose:
    - conditions: "{{ trigger.json.type == 'session.started' }}"
      sequence:
        - action: light.turn_on
          target: { entity_id: light.function_room }
          data: { brightness_pct: 25, transition: 1 }
    - conditions: "{{ trigger.json.type == 'session.ended' }}"
      sequence:
        - action: light.turn_on
          target: { entity_id: light.function_room }
          data: { brightness_pct: 100, transition: 2 }

```

The booth's JSON is available as `trigger.json`, so `trigger.json.data.mode` or `trigger.json.data.reason` work too. Home Assistant doesn't check BoothLedger signatures, so keep the webhook id private and `local_only: true` on. On Home Assistant versions before 2024.10, use the older keys: `trigger:` and `action:` as the section names, `platform: webhook` for the trigger and `service: light.turn_on` for each step.

D. Smart plug: fairy lights in a Santa's grotto

Turn a plug-in string of fairy lights or a spotlight on while a family is having their photo taken, and off afterwards. With a Shelly Plus Plug, add two **Shelly relay** triggers (GET):

Name	Address	Call it when
Grotto lights on	<code>http://192.168.1.70/rpc/Switch.Set?id=0&on=true</code>	Session starts
Grotto lights off	<code>http://192.168.1.70/rpc/Switch.Set?id=0&on=false</code>	Session ends

A Tasmota plug works the same way with **Other web address** and `http://192.168.1.71/cm?cmd=Power%20on / Power%20off`. Any device that can be switched by opening a web address works.

4. Webhooks from BoothLedger

Webhooks send your booth's moments to services on the internet: Zapier, Make, a CRM or your own server. BoothLedger sends them, not the booth, and every request is signed.

- Webhooks are set up **per account**, not per event: up to **10 webhooks**. Each one receives the moments you tick from every event you own. Use `event.id` or `event.name` in the payload to tell events apart.
- Moments from an event shared with you go to the event **owner's** webhooks.
- You can tick every **session moment**, plus **Photo uploaded to BoothLedger (with share link)**, which BoothLedger sends itself when a photo or video arrives.

Add a webhook

- 1 In BoothLedger go to **Automations > Webhooks** and click **Add webhook**.
- 2 Paste the **Address (https)** of the service, for example a Zapier "Catch Hook" address. Add an optional **Name** such as *Zapier – photo log*.
- 3 Under **Send it when**, tick the moments. *Session ends* and *Photo uploaded* are ticked for you.
- 4 Click **Create webhook**. The **Signing secret** appears: click **Copy**, store it in your receiving app, then **Done**.
- 5 Click **Send test** on the webhook's card. BoothLedger sends a `webhook.test` straight away and shows the answer, for example *Delivered — HTTP 200 in 312 ms* or *Failed: HTTP 401*. You can send up to 5 tests a minute.

Each webhook card also has **Turn off/Turn on**, **Edit** (address, name, moments), **Deliveries**, **Show secret**, **New secret** and **Delete** (which also removes its delivery log), plus the time of the last success and last failure.

Address rules

- It must start with `https://` and use the standard port (443) or 8443.
- It must be a public internet address. `localhost`, `.local` and `.internal` names and private, loopback or link-local IPs are refused, and every send checks the address again. For devices on the venue network, use **triggers** instead.
- No username or password in the address and no spaces.

Delivery, retries & the deliveries log

- Your server has **10 seconds** to answer (15 seconds at most for the whole request). Any **2xx** answer is a success. Anything else, including a redirect, a 4xx, a timeout or a connection error, is a failure. The response body is ignored, so answer quickly and do any slow work afterwards.
- A failed delivery is retried after **1 minute, 5 minutes, 30 minutes, 2 hours, 6 hours and 12 hours**: 7 tries over about 21 hours. After that it is marked *Failed*.

- **Deliveries** on the webhook card shows the latest 50: when, the moment, the result (*OK, Retrying — reason · next ... or Failed — reason*) and the number of tries. Click **Retry** on a failed delivery to send it again once your server is fixed. Test sends can't be retried: use Send test again. The log keeps 30 days.
- After **50 failed tries in a row** the webhook is turned off and shows *Turned off after 50 failed deliveries in a row*. Fix the receiver and click **Turn on**. While a webhook is off, nothing is queued for it.
- If one webhook has more than 2,000 deliveries waiting, new moments for it are dropped until the queue clears.
- **Offline booths:** the booth keeps its moments in an outbox on the PC (up to 500 moments, kept for 3 days) and forwards them when it's online again. It only forwards moments that one of your webhooks has ticked. So a moment can arrive late: use the `at` time, not the time it arrived. Moments can also arrive out of order.
- Moments from a **test-mode** event arrive with `"test": true`. Filter them out in your automations.

What BoothLedger sends

A `POST` with a compact JSON body and these headers:

```
Content-Type: application/json
User-Agent: BoothLedger-Webhooks/1
X-BoothLedger-Event: photo.uploaded
X-BoothLedger-Delivery: photo:3fld...
X-BoothLedger-Signature: t=1797098400,v1=5c1f...
```

Body (shown formatted)

```
{
  "v": 1,
  "id": "photo:3fld...",
  "type": "photo.uploaded",
  "at": "2026-12-12T18:02:11Z",
  "source": "cloud",
  "event": { "id": "9b7e...", "name": "Smith Wedding" },
  "session": "a41c...",
  "test": false,
  "data": {
    "photo_id": "3fld...",
    "share_url": "https://.../p/...",
    "media": "image"
  }
}
```

Field	Meaning
<code>v</code>	Payload version, currently <code>1</code> .

Field	Meaning
<code>id</code>	The moment's id: <code>booth:...</code> for booth moments, <code>photo:<photo id></code> for uploads, <code>test:...</code> for test sends. The same as the <code>X-BoothLedger-Delivery</code> header. It stays the same on every retry.
<code>type</code>	The moment, see Session moments .
<code>at</code>	When it happened, in UTC.
<code>source</code>	<code>booth</code> (forwarded by the booth) or <code>cloud</code> (sent by BoothLedger itself: uploads and tests).
<code>event</code>	The BoothLedger event's <code>id</code> and <code>name</code> .
<code>session</code>	The guest session id, so you can group moments and photos of one session.
<code>test</code>	<code>true</code> for test-mode events and test sends.
<code>data</code>	The moment's details (flat; text values up to 200 characters).

Ignore duplicates with the delivery id. A retry after a timeout can reach you twice. Store each `X-BoothLedger-Delivery` you have processed and answer `200` without doing the work again when you see it a second time.

The signing secret

- Each webhook has its own secret. It is shown when you create the webhook and again with **Show secret**, and it is stored encrypted.
- New secret** replaces it at once: requests signed with the old secret stop verifying, so update your receiver straight away.
- Booth triggers have their own secrets on the booth. The format and the check are the same.

Verify the signature

The `X-BoothLedger-Signature` header looks like `t=<unix seconds>,v1=<hex>`. To check it:

- 1 Read the **raw request body** as bytes, before any JSON parsing. Re-encoding the JSON changes the bytes and breaks the check.
- 2 Decode the secret as **base64url** (add `=` padding if your library needs it). The decoded bytes are the HMAC key.
- 3 Compute HMAC-SHA256 over `<t>.<raw body>` (the `t` value, a dot, then the body) and write it as lowercase hex.

- 4 Compare it with `v1` using a constant-time comparison.
- 5 Reject the request if `t` is more than 300 seconds (5 minutes) from your clock. This stops someone replaying an old request.

Python 3

```
import base64, hashlib, hmac, time

def verify_boothledger(secret: str, header: str, body: bytes, tolerance: int = 300) -> bool:
    """secret: the signing secret as shown. header: X-BoothLedger-Signature.
    body: the raw request body (bytes)."""
    try:
        parts = dict(p.split("=", 1) for p in header.split(","))
        t = int(parts["t"])
        sent = parts["v1"]
    except (KeyError, ValueError):
        return False
    if abs(time.time() - t) > tolerance:
        return False # too old (or the clocks are far apart)
    key = base64.urlsafe_b64decode(secret + "=" * (-len(secret) % 4))
    expected = hmac.new(key, f"{t}.".encode() + body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, sent)
```

Node.js 16 or newer

```
const crypto = require('crypto');

function verifyBoothLedger(secret, header, rawBody, tolerance = 300) {
    const parts = {};
    for (const p of String(header || '').split(',')) {
        const i = p.indexOf('=');
        if (i > 0) parts[p.slice(0, i)] = p.slice(i + 1);
    }
    const t = Number(parts.t);
    if (!Number.isInteger(t) || !parts.v1) return false;
    if (Math.abs(Date.now() / 1000 - t) > tolerance) return false;
    const key = Buffer.from(secret, 'base64url');
    const expected = crypto.createHmac('sha256', key)
        .update(`${t}.`).update(rawBody).digest('hex');
    const a = Buffer.from(expected), b = Buffer.from(parts.v1);
    return a.length === b.length && crypto.timingSafeEqual(a, b);
}
```

For booth triggers the timestamp comes from the booth PC's clock, so keep Windows' automatic time setting on.

A small receiver on your own server

Python (Flask)

```
import os
from flask import Flask, request, abort
```

```

app = Flask(__name__)
SECRET = os.environ["BOOTHLEDGER_WEBHOOK_SECRET"]
seen = set() # use a database table in production

@app.post("/boothledger")
def boothledger():
    body = request.get_data() # raw bytes: verify BEFORE parsing
    if not verify_boothledger(SECRET, request.headers.get("X-BoothLedger-Signature", ""), body):
        abort(401)
    delivery = request.headers.get("X-BoothLedger-Delivery", "")
    if delivery in seen:
        return "", 200 # already handled: say OK so it isn't retried
    seen.add(delivery)
    msg = request.get_json()
    if msg["type"] == "photo.uploaded" and not msg["test"]:
        print(msg["event"]["name"], msg["data"]["share_url"])
    return "", 204

```

Node.js (Express)

```

const express = require('express');
const app = express();
const seen = new Set(); // use a database in production

app.post('/boothledger', express.raw({ type: 'application/json' })), (req, res) => {
    const ok = verifyBoothLedger(process.env.BOOTHLEDGER_WEBHOOK_SECRET,
        req.get('X-BoothLedger-Signature'), req.body);
    if (!ok) return res.sendStatus(401);
    const delivery = req.get('X-BoothLedger-Delivery');
    if (seen.has(delivery)) return res.sendStatus(200);
    seen.add(delivery);
    const msg = JSON.parse(req.body);
    if (msg.type === 'session.ended' && !msg.test) {
        console.log(`${msg.event.name}: session over, ${msg.data.photos} photos`);
    }
    res.sendStatus(204);
});

app.listen(3000);

```

Run it behind HTTPS (for example nginx or Caddy with a certificate) on a public address, then add that address as a webhook.

Recipes: Zapier & Make

Zapier and Make can't check BoothLedger's signature. Treat their hook address like a password, and filter out `test` moments. Webhooks by Zapier is a premium Zapier app, so check your Zapier plan includes it.

Zapier: a Google Sheets row for every photo, with its share link

- 1 In Zapier, create a Zap. Trigger: **Webhooks by Zapier > Catch Hook**. Copy the webhook URL it

gives you.

- 2 In BoothLedger, **Automations > Webhooks > Add webhook**: paste the URL, tick only **Photo uploaded to BoothLedger (with share link)** and click **Create webhook**.
- 3 Give Zapier a real sample: take a photo on a booth with a **test-mode** event, wait for it to upload, then click **Test trigger** in Zapier. (Send test also reaches Zapier, but a `webhook.test` has no share link to map.)
- 4 Add **Filter by Zapier**: only continue if `Type` exactly matches `photo.uploaded` and `Test` is false.
- 5 Add **Google Sheets > Create Spreadsheet Row**. Make a sheet with columns *Time*, *Event*, *Session*, *Share link*, *Media* and map them to `At`, `Event Name`, `Session`, `Data Share Url` and `Data Media`.
- 6 Turn the Zap on. Every photo and video now lands in the sheet with a link the client can open.

Zapier: a Slack message when a booth starts its first session of the night

- 1 Create a Zap with **Catch Hook** and add a BoothLedger webhook that ticks only **Session starts**.
- 2 Filter: continue only if `Test` is false.
- 3 **Formatter by Zapier > Date / Time > Format**: turn `At` into a date like `YYYY-MM-DD` in your time zone.
- 4 **Storage by Zapier > Get Value** with the key `first-session-` plus `Event Id`.
- 5 Filter: continue only if the stored value does not exactly match the formatted date.
- 6 **Storage by Zapier > Set Value**: the same key, set to the formatted date.
- 7 **Slack > Send Channel Message**: "`Event Name` is live: first session started at `At` (`Data Mode`)."

Make: email each photo's link to the client as it uploads

- 1 In Make, create a scenario that starts with **Webhooks > Custom webhook**. Click **Add**, name it and copy the address.
- 2 In BoothLedger add a webhook with that address, tick **Photo uploaded to BoothLedger (with share link)**, and create it.
- 3 In Make click **Redetermine data structure**, then take a photo at a test-mode event so Make learns the fields.
- 4 Add a filter after the webhook: `type` equal to `photo.uploaded` and `test` equal to `false`.
- 5 Look up who to send to: for example **Google Sheets > Search Rows** in a sheet that lists *Event name* and *Client email*, matched on `event.name`.
- 6 Add **Gmail > Send an email** (or any email module) to that address with `data.share_url` in the message. Turn the scenario on.

The webhook carries one share link per photo or video. To send one email per session instead, collect the links by `session` (for example with a Make data store) and send them when you want.

5. Control API (local)

The control API lets other programs and devices control the booth: check its status, start a session, lock and unlock it, and add paid credits. It runs on the booth PC and needs no internet. It is **off by default** and its settings belong to that PC only.

Turn it on

- 1 On the booth open **Settings > Integrations > Control API** and switch on **Control API on**.
- 2 Under **Access token**, tap **Show** and copy the token. Send it with every request as the header `Authorization: Bearer <token>`. **New token** makes a new one and the old one stops working.
- 3 **Status** shows the address: *Programs on this PC: http://127.0.0.1:8643*. To use a phone or another device, switch on **Allow other devices on the network**; the status then also shows *Phones and other devices: http://<this PC's IP>:8643*.

The port is always **8643**. If another program already uses it, the status says *Port 8643 is already in use by another program*. Opening the address in a web browser shows a short notice and “unauthorized”: that’s normal, because the API is for apps, not browsers.

Security rules

- Every request needs the token. Keep it private: anyone with it can start sessions and add credits.
- Requests from web browsers (anything with an `Origin` header) are refused with `403`, and so are requests whose `Host` is a hostname instead of an IP address. Call the booth by its **IP address** (or `127.0.0.1` / `localhost` on the PC itself), never by a name like `booth-pc.local`.
- 10 wrong tokens from another device lock that device out for 10 minutes (`429`). Programs on the booth PC are never locked out.
- **Network mode is plain HTTP**. Anyone on the same Wi-Fi could read the token and add free credits. Only allow other devices on a private network you control, never on guest Wi-Fi.
- `/v1/unlock` only works from the booth PC itself.

Endpoints

Request	Body	Success	Refusals
<code>GET /v1/status</code>	—	<code>200</code> with <code>state</code> (<code>ready</code> , <code>busy</code> , <code>locked</code> , <code>not_ready</code>), <code>event</code> (<code>id</code> , <code>name</code>), <code>credits</code> , <code>credits_required</code> , <code>session</code>	—

Request	Body	Success	Refusals
POST /v1/session/start	<code>{"mode":"photo"}</code> — photo (default), boomerang or video	200 { <code>"ok":true,"mode":"photo"</code> }	409 <code>busy</code> , <code>not_ready</code> , <code>needs_payment</code> , <code>locked</code> , <code>bad_mode</code> (mode not available); 400 <code>bad_mode</code> (unknown mode), <code>bad_json</code>
POST /v1/lock	—	200 { <code>"ok":true,"locked":true</code> } : the booth shows its “paused” lock screen	409 <code>no_pin</code> : set a settings PIN first (Settings > Security)
POST /v1/unlock	—	200 { <code>"ok":true,"locked":false</code> }	403 <code>local_only</code> from another device
POST /v1/credits/add	<code>{"amount":1,"id":"tx-1234","source":"till"}</code> — amount 1–100 (default 1); id optional, up to 128 characters; source optional, up to 32 characters (default api)	200 { <code>"ok":true,"balance":3</code> }, or 200 { <code>"ok":true,"duplicate":true</code> } if that id was already credited	400 <code>bad_amount</code> , <code>bad_id</code> , <code>bad_json</code> ; 409 <code>balance_full</code> (the balance is capped at 1,000)

Every endpoint can also answer `401 unauthorized` (missing or wrong token), `403 forbidden` (browser request or hostname) and `429 too_many_attempts`. Bodies are JSON, up to 4 KB.

About `session/start` : it follows the same rules as a guest tap. It only works while the booth is idle on the Capture screen, never in import-only mode, and pay to play still applies (`needs_payment` if the credits or bundle don’t cover it). A `200` means the start was *requested*: the guest can still back out of a design or background choice. If you need certainty, watch for the `session.started` moment.

About `credits/add` : always send a unique `id` per payment (your till’s transaction number, for example). If your request times out and you send it again, the booth recognises the id and doesn’t add the credits twice.

Examples

curl (on the booth PC, or from another device with network access on)

```
# Check status
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:8643/v1/status

# Start a photo session
curl -s -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"mode":"photo"}' http://127.0.0.1:8643/v1/session/start

# Add one paid credit with a unique id
curl -s -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"amount":1,"id":"till-000123","source":"till"}' http://127.0.0.1:8643/v1/credits/add

# Pause the booth between bookings (from another device: use the booth's IP)
curl -s -X POST -H "Authorization: Bearer $TOKEN" http://192.168.1.30:8643/v1/lock
```

PowerShell on the booth PC

```
$h = @{ Authorization = "Bearer $env:BOOTH_TOKEN" }
$api = "http://127.0.0.1:8643"

Invoke-RestMethod -Uri "$api/v1/status" -Headers $h

# Lock now, unlock later (unlock works from this PC only)
Invoke-RestMethod -Method Post -Uri "$api/v1/lock" -Headers $h
Invoke-RestMethod -Method Post -Uri "$api/v1/unlock" -Headers $h

# Start a boomerang
Invoke-RestMethod -Method Post -Uri "$api/v1/session/start" -Headers $h `
  -ContentType "application/json" -Body '{"mode":"boomerang"}'
```

Ideas

- **A "Start" button on a phone.** iPhone Shortcuts (*Get Contents of URL*, method POST, header `Authorization`) or the Android app HTTP Shortcuts, pointed at `http://<booth IP>:8643/v1/session/start`. Needs *Allow other devices on the network* and a private network.
- **A Stream Deck on the booth PC.** Save the PowerShell lines above as small scripts and run them with the Stream Deck's *System > Open* action: one key to start, one to lock, one to unlock.
- **Pause during speeches.** Lock the booth before the speeches and unlock it afterwards.
- **Your own payment system.** A till, kiosk or Raspberry Pi bridge adds credits with `/v1/credits/add` after each payment.

6. Pay to play

Tip: set it up and try a paid session at home with your payment device before the event, so you know the reader, coins or QR code work end to end.

With pay to play, guests pay before each session. Payments arrive at the booth as **credits**, and a session only starts once enough credits are paid in. Until then the capture screen shows the ways to pay, such as “Tap your card to start” and/or a QR code.

Switch it on

Set it up on the booth in **Settings > Integrations > Pay to play**, or on BoothLedger in the event's **Integrations** tab, **Pay to play** card. The event settings sync both ways.

- 1 Switch on **Charge for sessions**.
- 2 Set **Credits per session**: what one go costs, in credits (1–20). Or add [packages](#).
- 3 Under **Ways to pay**, choose **Card reader / coin (tap)**, **QR code (Stripe)** or both. If no way to pay is on, the booth warns you that guests can't start a session.
- 4 On BoothLedger click **Save integrations**.
- 5 Test it: on the booth, **Add 1** gives a free credit, so you can try a full session. **Clear** empties the balance.

How credits work

- Credits are taken when the session really starts, after any design or background choice.
- If the session ends before the first photo (cancelled, an error or the app closed), the credits go back on the booth automatically, so the next guest can start.
- The balance (*Credits on this booth*) is saved on the PC, so a restart loses nothing. It is capped at 1,000 credits.
- **Upload Photo is never sold**. It's a tool for testing AI effects. With Charge for sessions on, it is free in test mode and hidden on the capture screen at a live event.

Packages & bundles

Instead of one price for any go, sell up to **3 packages**, for example *Photo* for 2 credits and *Photo + Boomerang* for 3.

- 1 Under **Packages** click **Add package**.
- 2 For each package choose what's included: a number of photos, boomerangs and videos (0–5 each), or **Any one** (one go of photo, boomerang or video; the guest chooses).

- 3 Set its **Credits** (1–20). With QR payments on, each package shows its price, for example “€7.50 by QR”.
- 4 For bundles, set **Time to finish a bundle** (0.5–15 minutes, default 2).

What guests see: with one package there's no choice. With two or three, the booth first shows *Choose your package* with big buttons and their prices; after picking one, the guest sees the ways to pay and a **Change** button. Once paid, only the start buttons included in the package are shown.

Bundles: the credits are taken when the first part starts. After it, a banner says what's left (for example “Boomerang included — tap Boomerang when you're ready”) with a countdown and a **Skip** button. The countdown pauses while a part is running. When it runs out, or the guest taps Skip, the rest is dropped. If the first part ends before its first photo, the credits are refunded to the booth and the bundle is cancelled.

A package is only offered if every mode in it has a start button at the event (for example, boomerang switched on). Settings warn you about a package guests won't be offered. Booths on older app versions (v1.0.247 and earlier) charge the first package's credits for every go.

Card reader & coin acceptor (tap)

Card readers and coin or note acceptors send credits to the booth as a key press. The booth counts each press of the **Payment device key** as one credit. Choose a key from F13 to F24, which no normal keyboard has.

Nayax VPOS Touch in pulse mode

- 1 In Nayax Core, set the device to **pulse** output and set the price so that one successful payment sends the number of pulses you want as credits.
- 2 Wire the pulse output to a **pulse-to-USB keyboard (HID) adapter** set to send **F13** per pulse, and plug it into the booth PC.
- 3 On the booth, set **Payment device key** to **F13** and **Credits per session** to match.
- 4 Make a test payment: the gate shows the credits arriving, then “Ready — press Start”.

Coin and note acceptors with a pulse output work the same way: one pulse, one credit.

- Key presses only count while the BoothLedger app is in front. Close EOS Utility and similar apps (the booth offers to).
- Presses closer together than 50 ms count as one, and at most 60 count per minute.
- Keep the PC's USB ports out of guests' reach: a programmable keyboard can send F13.
- Pulse mode is one-way. If a session fails after payment, the card payment isn't voided, but the booth keeps the credits so the next tap starts a session.

Card reader and coin payments go straight to your own Nayax account or cash box; BoothLedger takes nothing from them. Nayax charges its own fees.

QR payments with Stripe

No card reader needed: the booth shows a QR code, the guest scans it and pays on their phone with Apple Pay, Google Pay or a card, and the session starts once the payment arrives. The money goes to **your own Stripe account**.

- 1 On BoothLedger open the event's **Integrations** tab. In **Pay to play**, under *Ways to pay*, switch on **QR code (Stripe)**.
- 2 Click **Connect Stripe** and complete Stripe's sign-up (or sign in to an existing Stripe account). You come back to the Integrations tab. If Stripe still needs details, the button says **Finish Stripe setup**. When it's done you'll see ✓ *Connected* and *Payments in* your Stripe account's currency. You only connect once per account.
- 3 Set the **Price per credit**, in your Stripe currency. The lowest price is Stripe's minimum charge for that currency (for example €0.50 or £0.30); the highest is 1,000. A package costs its credits times this price.
- 4 Click **Save integrations**. QR payments and the price can also be changed on the booth (**Settings > Integrations > Pay to play**); until Stripe is connected, the booth's QR switch stays disabled with a hint.
- 5 Start the event live (**START LIVE EVENT**) and check the QR appears on the capture screen.

Payments always go to the **event owner's** Stripe account. On an event shared with you, the owner connects Stripe.

What the guest sees

- "Scan to pay €5.00" with "Apple Pay, Google Pay or card". Scanning opens a Stripe Checkout page with your business name. The QR asks for exactly the credits still missing.
- After paying, the booth shows "Payment received" and the guest presses Start.
- With only QR switched on, the gate shows "Getting the payment code ready..." while it loads, and "Payment isn't available right now — please ask a member of staff" if no QR can be shown.

Rules to know

- **Test mode takes no real payments.** In a test-mode event the booth says "QR payments start when you go live". Use **Add 1** to try a session.
- **Needs internet.** The QR shows only while the booth PC is online, Charge for sessions and QR payments are on, the event is on BoothLedger, the Capture screen is showing and the guest is short of credits. Offline, card readers and coins keep working.
- **Automatic refund.** If a guest pays but the booth doesn't collect the payment within **15 minutes** (for example the PC crashed or went offline), BoothLedger refunds the guest in full, including BoothLedger's fee.
- If a paid session fails before its first photo, the credits go back on the booth for the next go; the guest's money isn't refunded automatically. You can refund it from the [Payments](#) page.

- BoothLedger keeps **3.7%** of each QR payment, plus VAT where it applies, and Stripe's own fees apply too. See [Fees & VAT](#).

PIN protection for paying events

A booth that charges guests must not let a guest open Settings and switch payments off or add free credits. So while an event is **live** and **Charge for sessions** is on, the booth needs a **settings PIN**:

- If no PIN is set, the booth asks: "This event charges guests — protect it with a PIN". Set a 4-digit PIN on the on-screen pad, or tap **Turn off Charge for sessions**.
- Tapping **START LIVE EVENT** asks you to set a PIN first, if none is set. Cancel keeps the event in test mode.
- Once protected, the booth goes to fullscreen kiosk mode, and the Charge for sessions switch, **Add 1** and **Clear** ask for the PIN.
- In test mode there's no PIN requirement, so you can set things up freely.

7. Payments, fees & invoices

The Payments page

In BoothLedger, **Payments** (in the main menu, once QR payments are available on your account) shows what guests paid by QR code.

- Pick a period (*This month, Last month, This year, All time*) and an event, or all events. *Include test payments* adds payments made while testing.
- Totals: **Takings, Refunded, BoothLedger fee** (with VAT shown separately where it applies) and **You receive**, which is before Stripe's own processing fees.
- **By event** and **By month** breakdowns, and the list of **Payments** with their status: *Paid, Paid (not yet used), Refunded* (by you, in Stripe, or because the booth didn't collect it), *Failed*, or *Refund failed – please refund it*.
- Refund a payment from the list. A refund returns BoothLedger's fee too.
- **Download CSV** exports the payments for your bookkeeping.

How you get paid

QR payments are charged on **your own Stripe account**, so the money is yours from the start. BoothLedger's fee is taken by Stripe at the moment of payment, and Stripe pays the rest out to your bank account on your Stripe payout schedule. Payouts, Stripe's fees and disputes are managed in your Stripe Dashboard. Card reader and coin payments never pass through BoothLedger.

Fees & VAT

BoothLedger keeps **3.7% of each QR payment**. The fee is **VAT-exclusive**: where VAT applies, it is added on top. BoothLedger is based in Ireland, so the rules are:

Your billing details	VAT on the fee
Business in Ireland	Irish VAT at 23% added
In another EU country, with a VAT number on your account	No VAT: reverse charge
In another EU country, without a VAT number	Irish VAT at 23% added
Outside the EU	No VAT (outside the scope of Irish VAT)
No billing country on file	Irish VAT at 23% added; add your billing details to fix this

The VAT treatment is fixed for each payment when the guest pays, using your billing details at that time. On BoothLedger and on the booth the fee is shown as "3.7% (+ VAT)" when VAT is added for you.

Example: a €5.00 payment from an Irish business. Fee: 3.7% of €5.00 = €0.185, rounded to €0.19. VAT: 23% of €0.19 = €0.04. BoothLedger keeps €0.23 in total; Stripe's own processing fee is separate.

Monthly invoices

- On the **1st of each month** you get one invoice for the previous month's BoothLedger fees, per currency, if there were any. Refunded payments are left out.
- It is emailed to you with a PDF, and listed under **Invoices** on the Payments page, where you can view or download it.
- The fee was already taken by Stripe, so there's nothing more to pay. The invoice is your VAT record.
- Invoice numbers look like `BLQ-2026-0001`.

8. Troubleshooting

Why doesn't my trigger fire during a session?

Check that Triggers on (or Triggers enabled on BoothLedger) is switched on, the trigger itself is on, and the right moments are ticked. Then tap Send test on the booth. A test that fails with "couldn't reach the device" or "no answer in time" means the booth can't reach that address: check the device's IP, that it is on the same network as the booth PC, and that the address works when opened on the booth PC. Changes made on BoothLedger take about 45 seconds to reach the booth.

Why did my light or relay fire late, or not at all, when several moments happened together?

The booth runs at most 4 calls at once and drops any cue that has waited more than 30 seconds, so lights never fire long after the moment. Slow devices that take close to the 3-second timeout can hold up the queue. Give devices a fixed IP and avoid ticking moments you don't need.

Why does my receiver reject the signature with a 401?

The usual causes are: the body was parsed and re-encoded before checking (always use the raw bytes), the secret wasn't base64url-decoded before use, the wrong secret was used (each webhook and each booth trigger has its own), the secret was replaced with New secret, or the clocks are more than 5 minutes apart. If a booth trigger's address was moved to another device, the booth sends it unsigned until you create a new secret.

Why won't BoothLedger accept my webhook address?

Webhook addresses must start with https://, use port 443 or 8443, and point at a public internet address. Local names (localhost, .local, .internal) and private IP addresses are refused. To call a device on the venue network, use a trigger on the booth instead.

Why was my webhook turned off?

After 50 failed tries in a row, BoothLedger turns the webhook off and shows "Turned off after 50 failed deliveries in a row". Open Deliveries to see the errors, fix the receiver, click Turn on and use Retry on the failed deliveries you still need.

Why do I receive the same moment twice?

If your server answers slowly or the connection drops, BoothLedger can't tell whether the delivery arrived and sends it again. Store the X-BoothLedger-Delivery id of every request you handle and answer 200 without repeating the work when the same id comes back.

Why are webhook moments arriving late?

When the booth is offline, it keeps its moments (up to 500, for 3 days) and forwards them when it is online again. Photo uploads also wait for the internet. Use the at field for when the moment happened.

Why isn't the QR code showing on the booth?

The QR only shows when all of these are true:

- the event is live, not in test mode (test mode says "QR payments start when you go live")
- the booth PC is online
- Charge for sessions and QR code (Stripe) are on, and the event is saved on BoothLedger
- Stripe is connected and set up (Connected on the Integrations tab)
- a price per credit is set and is at least Stripe's minimum for your currency
- the Capture screen is showing and the guest is short of credits

A guest paid by QR but didn't get a session. What happens?

The booth collects a paid QR payment as soon as it can, including after a restart or when it comes back online. If it can't collect it within 15 minutes, BoothLedger refunds the guest in full automatically. You can see each payment's status on the Payments page and refund it yourself there.

Why does the booth say "Payment isn't available right now"?

Card reader / coin (tap) is switched off and the QR code can't be shown (offline, Stripe not connected or an error), or none of your packages can be offered at this event, for example a package that includes a boomerang when boomerang is switched off. Switch on another way to pay or fix the package.

Why does the control API answer unauthorized, forbidden or too_many_attempts?

401 unauthorized means the Authorization: Bearer header is missing or the token is wrong (New token replaces it). 403 forbidden means the request came from a web browser or used a hostname: call the booth by its IP address. 429 too_many_attempts means 10 wrong tokens came from that device; wait 10 minutes. From another device, also check Allow other devices on the network is on.

Can a guest switch payments off on the booth?

Not on a protected booth. While a paying event is live, the booth needs a settings PIN, runs in fullscreen kiosk mode, and asks for the PIN before Charge for sessions, Add 1 or Clear can be used.

Still stuck? Email info@photoboothguys.ie or use the help assistant in BoothLedger. For an overview of running an unattended booth, see [Pay-Per-Use Photo Booth Software](#).